

IVOLT

Privacy-First Technology & Security



The Menu Subsystem

Developer Guide: Structure, Usage & Expansion

File: Menu_System.cs

Namespace: IVolt.Core.Email.Gmail

Document version 1.0 - June 30, 2026

Contents

1. Purpose & Design Philosophy
2. Component Map
3. The Menu Class
4. The Menu_Item Type
5. Registering Items — the Add API
6. The Render / Input Loop
7. Conditional Visibility
8. Worked Example 1 — Edit Configuration
9. Worked Example 2 — Retry Logged Errors
10. The Failure_Log Companion
11. Wiring a Menu into the Engine
12. Expansion Patterns
13. Nested & Dynamic Menus
14. Conventions & Pitfalls
15. Quick Reference

1. Purpose & Design Philosophy

Menu_System.cs provides a small, extensible console menu framework for the Gmail IMAP Archiver. Its goal is that adding a new operator capability is a single registration call — never an edit to the rendering or input-handling logic. You supply a label and an action; the framework owns the loop, the numbering, input validation, and error isolation.

- **Open for extension, closed for modification** — add items with Add(...); the loop never changes.
- **Action-oriented** — each item is just a label bound to an Action (or a Func<bool> for status).
- **Fail-safe** — an exception in one item is caught and reported; the menu keeps running.
- **Conditionally visible** — items can hide themselves when they don't apply.

2. Component Map

Type	Kind	Role
Menu	class	Holds items; renders; runs the input loop until Back/Exit.
Menu_Item	class	One entry: Label, Action, optional Visible predicate.
Failure_Log	class	Persistent failure record read by the retry item (companion).
Archive_Operations	static class	The two fully-coded items + a BuildMenu() factory.

3. The Menu Class

A Menu is constructed with a title and, optionally, redirected I/O:

```
var menu = new Menu("OPERATIONS");
// optional: new Menu("OPERATIONS", customWriter, customReader);

menu.Title      = "OPERATIONS"; // heading shown above items
menu.BackLabel  = "Back";       // label for the 0 option
```

By default it reads and writes the console. Passing a TextWriter/TextReader lets you unit-test a menu or drive it from a script — the loop never calls Console directly when overrides are supplied.

4. The Menu_Item Type

```
public sealed class Menu_Item
{
    public string Label { get; set; } // shown next to its number
    public Action Action { get; set; } // invoked on selection
    public Func<bool> Visible { get; set; } // optional; null = always shown
}
```

You rarely construct `Menu_Item` directly — the `Add` overloads build it for you. It is public so advanced callers can pre-build item lists if desired.

5. Registering Items — the Add API

There are two `Add` overloads, both fluent (they return the `Menu`):

Void action:

```
menu.Add("Show Summary", () => ShowSummary(store));
menu.Add("Edit Configuration", () => Archive_Operations.EditConfiguration(email));
```

Status action (Func<bool>) — prints a done/issues line automatically:

```
menu.Add("Retry Logged Errors",
        () => Archive_Operations.RetryLoggedErrors(cfg, store));
// returns true -> " OK Done."
// returns false -> " ! Completed with issues."
```

Because `Add` returns the `Menu`, registrations chain:

```
new Menu("OPERATIONS")
    .Add("Edit Configuration", () => EditConfiguration(email))
    .Add("Retry Logged Errors", () => RetryLoggedErrors(cfg, store))
    .Show();
```

6. The Render / Input Loop

`Show()` runs until the user enters 0. Each pass:

Step	Behavior
1. Filter	Build the visible list (items whose <code>Visible</code> predicate is null or true).
2. Render	Print the title, number each visible item 1..N, then '0. Back'.
3. Read	Read a line; '0' returns from <code>Show()</code> .
4. Validate	Non-numeric or out-of-range input prints a notice and re-prompts.
5. Invoke	Call the chosen item's <code>Action</code> inside <code>try/catch</code> .
6. Isolate	An exception is caught and printed; the loop continues.

Numbering is dynamic

Items are numbered against the **visible** list each pass, so hidden items never leave gaps. Don't hard-code selection numbers in help text — the framework owns them.

7. Conditional Visibility

Pass a predicate as the third Add argument to show an item only when it applies. Example: hide the retry item when the failure log is empty.

```
menu.Add("Retry Logged Errors",
    () => Archive_Operations.RetryLoggedErrors(cfg, store),
    visible: () => new Failure_Log(store.ArchiveFolder,
        store.AccountKey).UnresolvedDistinct().Count > 0);
```

The predicate is evaluated every render pass, so visibility tracks live state — as soon as all errors are resolved, the item disappears on the next redraw.

8. Worked Example 1 — Edit Configuration

Archive_Operations.EditConfiguration demonstrates a self-contained item that loads state, edits it, and persists it. Its shape is a template for any 'edit a thing' item:

- **Resolve target** — prompt for the email if not supplied; derive the config path.
- **Load** — DPAPI-unprotect and deserialize into the typed container.
- **Edit loop** — show fields, accept one change at a time, track a dirty flag.
- **Secrets** — masked display and masked entry; Enter keeps the existing value.
- **Persist** — on Save, re-serialize and DPAPI-protect back to the same path.

Reusable pattern

Load -> edit-loop-with-dirty-flag -> confirm-on-discard -> save. Any settings editor in the app can follow this exact structure.

9. Worked Example 2 — Retry Logged Errors

Archive_Operations.RetryLoggedErrors demonstrates an item that consumes persistent state and coordinates with the fetch engine:

- **Read** the failure log's unresolved, de-duplicated UIDs.
- **Confirm** with the operator before acting.
- **Group by folder** so each mailbox opens once.
- **Reprocess** each UID via Gmail_Fetch_Engine.RetryOne (same defensive path as a normal fetch).
- **Mark resolved** the UIDs that now succeed, so repeated runs converge.
- **Report** recovered vs. still-failing counts; return true only if all recovered.

This item shows the recommended way to reach into a heavier subsystem (the fetch engine) from a menu action without duplicating its logic.

10. The Failure_Log Companion

Failure_Log is a per-account, append-only JSON-lines file next to the archive. The fetch engine writes to it when it gives up on a message; the retry item reads and updates it.

Member	Purpose
Record(uid, validity, folder, reason)	Append one failure entry.
ReadAll()	All entries, tolerant of blank/garbled lines.
UnresolvedDistinct()	Distinct unresolved UIDs, latest reason kept.
MarkResolved(uids)	Rewrite the log marking the given UIDs resolved.
LogPath	Full path to the log file.

Naming pitfall

Expose the path as **LogPath**, not **Path** — a property named Path shadows System.IO.Path and breaks Path.Combine(...) inside the class. This is a real compile error to avoid.

11. Wiring a Menu into the Engine

The archiver's main menu is hand-rolled; to surface the operations submenu, add one case that builds and shows it:

```
// in Email_Archiver_Engine.MainMenu(...)
case "7":
    Archive_Operations.BuildMenu(cfg, store).Show();
    break;

// BuildMenu registers both coded items and returns a ready Menu:
public static Menu BuildMenu(Configuration_Definition_Container cfg,
    Archive_Store store)
{
    var menu = new Menu("OPERATIONS");
    menu.Add("Edit Configuration File", () => EditConfiguration(cfg?.email));
    menu.Add("Retry Errors Found in Logs", () => RetryLoggedErrors(cfg, store));
    return menu;
}
```

12. Expansion Patterns

To add a new capability, write a method and register it. Three shapes:

A. Simple action

```
static void ExportManifestCsv(Archive_Store store) { /* ... */ }
menu.Add("Export Manifest to CSV", () => ExportManifestCsv(store));
```

B. Status-returning action

```
static bool VerifyHashes(Archive_Store store) {
    // recompute SHA-256 for each record; return false if any mismatch
    return true;
}
menu.Add("Verify Archive Integrity", () => VerifyHashes(store));
```

C. Conditionally visible action

```
menu.Add("Rebuild Manifest from Tree",
    () => store.RebuildManifestFromTree(),
    visible: () => store.Manifest.message_count == 0);
```

Keep actions thin

A menu action should orchestrate, not implement. Put real work in a dedicated method or subsystem and have the action call it — this keeps items testable and reusable outside the menu.

13. Nested & Dynamic Menus

A menu item's action can itself build and Show() another Menu, giving arbitrary nesting. Returning from the inner Show() drops back to the outer menu automatically.

```
menu.Add("Maintenance", () =>
{
    new Menu("MAINTENANCE")
        .Add("Rebuild Manifest", () => store.RebuildManifestFromTree())
        .Add("Verify Integrity", () => VerifyHashes(store))
        .Show();
});
```

For fully dynamic menus (items known only at runtime), loop over your data and call Add in a builder method before Show — for example, one item per discovered plugin.

14. Conventions & Pitfalls

Do	Instead of
Register with Add(...)	Editing the Show() loop to add a branch.
Return bool for pass/fail items	Printing your own OK/fail line (Add does it).
Put work in a named method	Inlining large logic in the lambda.

Do	Instead of
Name the path property LogPath	Naming it Path (shadows System.IO.Path).
Use ASCII in console strings	Fancy glyphs that can trip file-encoding builds.
Let exceptions bubble to the loop	try/catch in every action (the loop already isolates).

15. Quick Reference

```
// Build
var menu = new Menu("TITLE");

// Register (void)
menu.Add("Label", () => DoThing());

// Register (status)
menu.Add("Label", () => DoThingReturningBool());

// Register (conditional)
menu.Add("Label", () => DoThing(), visible: () => condition);

// Run
menu.Show(); // loops until user enters 0

// Factory for the two coded items
Archive_Operations.BuildMenu(cfg, store).Show();
```

IVolt LLC - Privacy-First Technology & Security. Confidential; for internal developers. Menu_System.cs, namespace IVolt.Core.Email.Gmail.